can’t help you. In section 24.5.1 you’ll see the approach you need to take to handle these situations and to apply authorization inside your endpoints.

WARNING Be careful when exposing the integer ID of your entities in the URL, as in listing 24.13. Users will be able to edit every entity by modifying the ID in the URL to access a different entity. Be sure to apply authorization checks, or you could expose a security vulnerability called *insecure direct object reference* (IDOR). You can read more about IDOR at http://mng.bz/QPnG.

24.5.1 Manually authorizing requests with IAuthorizationService

All of the approaches to authorization so far have been *declarative*. You apply the `[Authorize]` attribute, with or without a policy name, and you let the framework take care of performing the authorization itself.

For this recipe-editing example, you need to use *imperative* authorization, so you can authorize the user after you’ve loaded the `Recipe` from the database. Instead of applying a marker saying “Authorize this method,” you need to write some of the authorization code yourself.

DEFINITION *Declarative* and *imperative* are two different styles of programming. *Declarative programming* describes what you’re trying to achieve and lets the framework figure out how to achieve it. *Imperative programming* describes how to achieve something by providing each of the steps needed.

ASP.NET Core exposes `IAuthorizationService`, which you can inject into any of your services or endpoints for imperative authorization. The following listing shows how you could update the Edit.cshtml Razor